

KALKAR HACKATHON 2026

Hackabike

The smart bike station that guides you to a free slot
and guards your bike until you come back.

THE PROBLEM

A bike rack tells you nothing and protects nothing.

Where is a free spot?

You only find out when you are standing in front of it.

Is my bike still safe?

A lock slows a thief down, but nobody notices.

Who parked here?

There is no link between a bike and its owner.



OUR SOLUTION

One station that knows every slot and every owner.



Access

NFC chip or phone app.
Only accounts get in, one bike each.



Guidance

Your slot lights up, the LCD greets you, the gate opens.



Protection

A bike leaves without its owner? Alarm, and the station locks.



LIVE DEMO

One rider, three moments

1 • Park

1. Tap chip or press *Park*
2. LCD "Park at Slot 2", slot blinks blue
3. Gate opens
4. Bike in: "Secured", gate closes

2 • Pick up

1. Tap again
2. Your slot blinks blue
3. Gate opens, take your bike
4. Slot empty: gate opens 5 s to leave

3 • Theft

1. Secured bike leaves, no pickup
2. Slot blinks red, gate shuts
3. Every chip and app refused
4. Admin clears it on the dashboard



What makes it different

One path for chip and phone

The app sends the same tap as a chip, so both follow exactly the same rules.

Alarms without false alarms

Only bikes secured through a reservation are guarded. Test objects and broken sensors never trigger.

The station defends itself

A theft locks every entrance until an admin clears it, even across restarts.

It learns when it gets busy

An hourly occupancy forecast from real sensor history, in German, English and Dutch.

Architecture

Phone

Web app, reached
through a Pangolin tunnel

Raspberry Pi 4

Mosquitto · Node-RED ·
PostgreSQL · web app, in
Docker Compose

Own Wi-Fi **network**

ESPs talk MQTT to the Pi,
with no route to the
school LAN

ESP32 #1

LED strip for all 4 slots,
distance sensors slot 1 +
2

ESP32 #2

NFC reader, LCD, gate
servo, distance sensors
slot 3 + 4

Node-RED **decides**

The ESPs only measure
and execute

TECHNICAL IMPLEMENTATION

Hardware

Part	Model	Job
Server	Raspberry Pi 4 (4 GB)	broker, logic, database, web, Wi-Fi AP
Controllers	2 × ESP32 DevKit	sensors, LEDs, entrance
Slot sensors	4 × Grove Ultrasonic	bike present, every 200 ms
Slot lights	WS2812B LED strip	free · occupied · reserved · alarm
Access	PN532 NFC reader	chip taps
Display	Grove LCD RGB 16×2	greeting, slot, colour-coded status
Gate	Tower Pro MG90S servo	opens and closes the entrance

TECHNICAL IMPLEMENTATION

Every bike has a lifecycle

reserved

slot blinks blue, 5 min to arrive

parked

"Secured", gate closes

done

picked up by its owner

reserved → expired

no bike within 5 minutes, the slot is free again

parked → alarm

bike leaves without pickup, the station locks until an admin clears it

Debounced

a slot only changes after 5 equal readings

Built for the real world

Debounced sensors

Median filter on the ESP, 5 equal readings in Node-RED.

No echo $\neq$ empty

A broken sensor reports 999 and is ignored, never "bike gone".

State survives restarts

Alarms and reservations are stored on disk; a reboot cannot unlock.

Honest app feedback

The app waits for the station's answer before it says "open".

Isolated ESP network

The ESPs reach the broker and nothing else.

Load split

Two ESPs: every slot reports every 200 ms instead of every second.

LIVE DATA FROM THE STATION

It runs, and it records everything

483k

sensor readings stored

33

parking sessions, 31
completed

84

taps, 17 of them from
the app

8.5 s

average from tap to
"Secured"

Tested in code and on the station

14 logic scenarios, all passing

- unknown chip is refused
- park, secure, pick up, exit gate 5 s
- second bike per account refused
- sensor without echo: no alarm
- theft: alarm, gate shut, all taps refused
- admin reset reopens the station

On the real hardware

- every firmware validated with ESPHome
- unknown-chip tap and servo 90° → 0° live
- Pi reboot: everything back in about a minute
- every deploy: checksum check and backup first
- bugs found from the tap log in the database

OPEN SOURCE & DOCUMENTATION

Built on open source, documented in the open

ESPHome · Node-RED · Eclipse Mosquitto · PostgreSQL · Docker Compose · Node.js / Express · FlowFuse Dashboard

56

commits

4

contributors

8

READMEs

1,200+

lines of docs

What a hackathon build still lacks

Demo security

The broker and the Node-RED editor have no login yet. The steps to lock them down are documented.

NFC reader after a reset

It only recovers after a power cycle. Fine on power-up, annoying after an update.

Sound is designed, not built

Melody and alarm beep are planned, but we could not get an amplifier.

Four slots

A prototype size. More slots mean more sensors and LEDs, not a new design.



NEXT STEPS

From prototype to product

1. Logins for broker and Node-RED, secrets out of the repository
2. A transistor that power-cycles the NFC reader on boot
3. Entrance sound: welcome melody and alarm beep
4. More slots: one more ESP per two slots, same logic



Thank you

Questions? Try it yourself: tap a chip at the station.